Part A Evaluation Report: Comparing C and Python Sorting Implementations

1. Introduction

This report evaluates the tutor-provided C program (bubsort2.c) and Python program (bubsort.py) after modification. Both programs read the CPUPI.txt data set, while the Python implementation uses a more efficient sorting algorithm. The report describes the changes, justifies the algorithm choice using Big-O analysis, presents the available timing results, answers the assignment questions, and considers relevant professional, legal and ethical issues.

2. The Data Set and Original Programs

CPUPI.txt contains digits of Pi arranged in groups of ten. When each group is parsed as an integer, any leading zeros are removed. The report records 838,861 integer values, each containing up to ten digits. Verify the file's actual contents and stated decimal coverage before submission. The task is to sort the values into ascending numerical order.

The original C program sorted a hard-coded array of eleven integers and printed the result. The original Python program used a similarly small hard-coded list. Neither program read an external data file.

3. Modifications Made

3.1 C Program (bubsort2.c)

The bubble-sort function was retained unchanged, as required. The data-reading and output mechanisms were modified as follows:

1. Opens CPUPI.txt and checks that the file is available.

2. Makes a first fscanf pass to count the values; the data file spans 92,275 lines, so a fixed-size array is unsuitable.
3. Allocates sufficient memory using malloc.
4. Rewinds the file and reads the values into the allocated array.
5. Uses long long rather than int because ten-digit values can exceed the range of a signed 32-bit integer; the associated format specifiers are updated to %lld.
6. Times the sorting operation with clock().
7. Accepts an optional command-line argument to limit the sort to the first N values, allowing the required data sizes to be tested. Without the argument, it sorts the full set.
8. Prints a preview of the first and last ten values and verifies that the result is in ascending order, rather than printing every value.
9. Frees allocated memory before exiting.

3.2 Python Program (bubsort.py)

10. Reads CPUPI.txt and converts each whitespace-separated token to an integer, with checks for a missing file and non-numeric input.
11. Retains the original bubble sort for comparison, but uses Python's built-in list.sort() method for the main data set. This method uses Timsort.
12. Measures sorting time with time.perf_counter().
13. Prints the first and last ten sorted values, checks that the output is ascending, and writes all 838,861 values to sorted_data_full.txt.

4. Algorithm Selection and Justification

Bubble sort repeatedly compares adjacent values and swaps them when they are out of order. The tutor-provided implementation has no early-exit flag, so it performs $n(n-1)/2$ comparisons, giving $O(n^2)$ time complexity (Cormen et al., 2022). For n = 838,861, this is

approximately 3.52×10^{11} comparisons. This quadratic growth helps explain why runtimes rise sharply as the input grows (Knuth, 1998). Bubble sort remains in the C program because the assignment requires the original algorithm to be preserved there.

The Python implementation uses Timsort through list.sort(). Timsort is a stable, adaptive sorting algorithm that identifies naturally ordered runs and merges them efficiently. Its average and worst-case time complexity is O(n log n), while already ordered input can be processed in O(n) time. Its auxiliary memory use can be O(n) (Python Software Foundation, 2026). For this data size, $n \log_2 n$ is approximately 1.65×10^7. This is a useful order-of-growth comparison, not an exact count of operations performed by Timsort.

Insertion sort and selection sort have $O(n^2)$ time complexity in their standard forms. Merge sort has O(n log n) time complexity but would require a separate implementation. Quicksort is often efficient on average, but some implementations have $O(n^2)$ worst-case behaviour. Counting and radix sorts can be useful for bounded integer keys, although they would add implementation complexity that is unnecessary for this task. Python's maintained built-in sorting method was therefore selected for its performance, reliability and simplicity.

5. Experimental Method

Measurements were taken on a Windows 11 64-bit machine. The C program was compiled with Microsoft Visual Studio 2022 (MSVC cl.exe v14.44, x64, /O2), using the Visual C++ build-tools environment. Python 3.12.10 (CPython) was used for the Python tests. The timers measured the sorting operation only, excluding file reading and console output: clock() in C and time.perf_counter() in Python (Microsoft, 2026; Python Software Foundation, 2026).

The required input sizes were 100,000, 200,000, 500,000 and 800,000 values. Each test used the first N values from the same file so that the implementations were compared on corresponding input data. The efficient Python sort was also tested on all 838,861 values, and the C program was run on the full set. The figures below are single-run results; timing variation may occur because of background processes and other system conditions.

Because bubble sort has $O(n^2)$ time complexity, running Python bubble sort on hundreds of thousands of values was impractical. Its larger-input times are estimates based on smaller tests and the observed quadratic trend. These estimates are marked with '~' and are not direct measurements. The Python bubble-sort observations were 0.0298 seconds for 1,000 values, 0.8057 seconds for 5,000, 3.907 seconds for 10,000, and 19.096 seconds for 20,000. These limited measurements are consistent with rapid quadratic growth, although repeat runs would improve confidence.

The C runtime increased by about 8.8 times between 100,000 and 200,000 values, rather than the fourfold increase expected under a simple quadratic model. Cache effects, processor frequency changes and measurement variability may contribute, but these causes were not independently tested. The results should be interpreted as observations from one machine rather than universal performance guarantees.

6. Results

The table distinguishes direct measurements from estimates. The Python bubble-sort values at larger sizes are extrapolations. The 800,000-value Python Timsort value is approximate; label it as measured only if a recorded run confirms it.

Data size	C bubble sort	Python bubble sort	Python Timsort
100,000	4.115 s (measured)	~477 s (estimated)	0.0174 s (measured)
200,000	36.211 s (measured)	~1,910 s (estimated)	0.0393 s (measured)
500,000	249.627 s (measured)	~11,940 s (estimated)	0.1231 s (measured)
800,000	645.251 s (measured)	~30,600 s (estimated)	~0.23 s (verify whether measured or estimated)
838,861 (full set)	711.551 s (measured)	~33,600 s (estimated)	0.2394 s (measured)

The reported results show three main patterns. First, compiled C bubble sort is substantially faster than Python bubble sort when the same quadratic algorithm is used. At 100,000 values, the reported C time is 4.115 seconds compared with an estimated 477 seconds for Python bubble sort. Second, Python's Timsort is much faster than C bubble sort: 0.0174 seconds compared with 4.115 seconds at 100,000 values. Third, the difference is especially large on the full data set, where the reported times are 0.2394 seconds for Timsort and 711.551 seconds for C bubble sort.

These comparisons should be interpreted carefully. The Python bubble-sort figures are estimates, not measurements, and the benchmark was not repeated. The results illustrate the likely scale of the algorithmic difference, but they do not establish precise, universal speed ratios.

6.1 Answers to the Assignment Questions

Were you able to sort the data in Python faster than the compiled bubble sort in C?

Yes. Based on the reported timings, Python's Timsort completed the 100,000-value test in 0.0174 seconds, compared with 4.115 seconds for C bubble sort. On the full set, the reported times were 0.2394 seconds for Timsort and 711.551 seconds for C bubble sort.

Did it work for all the data sizes listed?

The report records successful sorting and ascending-order verification for the tested sizes. Before submission, confirm that the program outputs and saved sorted file substantiate each required size, especially the 800,000-value Timsort result if that figure is retained in the table.

7. Discussion: Comparing the Two Languages

7.1 The algorithm effect and the language effect

The results suggest that algorithm choice has a larger effect than the language difference in this experiment. With bubble sort used in both programs, the compiled C implementation is reported to be faster than Python's bubble sort. Within Python, replacing bubble sort with Timsort substantially reduces sorting time. These are implementation-level comparisons rather than perfectly controlled experiments: runtime environments, compiler optimisation and implementation details can also affect performance. The overall pattern is consistent with the difference between $O(n^2)$ and $O(n \log n)$ growth.

7.2 What each language made easier

The C version required dynamic memory allocation, a second pass through the input file, appropriate format specifiers for long long values, memory release and configuration of the compiler toolchain. Compiler warnings also required review. The result provides direct control over memory and a native executable, but it involves more implementation and build steps.

The Python version required less code for reading the data and sorting it. The built-in list.sort() method delegates the sorting work to a highly optimised implementation and avoids manual memory management. Python's dynamic typing and direct execution can make development and testing quicker, although some errors may only become apparent at run time. These are general tendencies rather than guarantees for every project.

7.3 Practical guidance

For production data processing, algorithm choice should be considered before low-level optimisation. Standard-library sorting functions are usually preferable to hand-written quadratic sorting for large data sets. Bubble sort remains useful for teaching and illustrating algorithmic complexity, but it is unsuitable for this input size when more efficient alternatives are available.

7.4 Lessons for software development

14. Select an algorithm that suits the size and characteristics of the input before attempting low-level optimisation.
15. Use established standard-library functions when they meet the requirements, reducing implementation effort and the risk of defects.
16. Consider data characteristics, because adaptive algorithms such as Timsort can benefit from existing order in the input.

17. Measure performance under documented conditions and distinguish direct measurements from estimates.

8. Professional, Legal and Ethical Considerations

Professional competence and tool selection

Computing professionals should select methods and tools appropriate to the problem and support their decisions with evidence. For large inputs, choosing a maintained O(n log n) sorting implementation instead of an O(n^2) algorithm can improve performance and reduce unnecessary processing (Bentley and McIlroy, 1993).

Honest reporting and reproducibility

Performance claims should be reproducible. Recording the operating system, compiler version and flags, Python version and timing functions helps readers understand the test conditions. Estimated values must be clearly distinguished from measured results. Repeated runs and reporting an average or median would improve the reliability of future tests.

Referencing and academic integrity

Academic and technical sources support the discussion of algorithm complexity, sorting and software tools. Borrowed ideas and documentation should be cited in accordance with the university's required style. Any generative-AI assistance should be disclosed in line with the institution's academic-integrity policy.

Professional codes of conduct

The BCS Code of Conduct emphasises professional competence, honesty and responsibility to the public (BCS, 2022). In this exercise, these principles are reflected in

documenting the method, checking the output and reporting limitations rather than presenting estimates as measured results.

Intellectual property and licensing

Python, Microsoft development tools and the cited textbooks are governed by their respective licences and copyright conditions. In professional work, developers should check the applicable terms before distributing software, reusing code or reproducing copyrighted material.

Data protection

CPUPI.txt contains digits of Pi rather than personal data. However, a similar file-processing program used with customer information would need to be assessed against applicable data-protection requirements, including the UK GDPR and Data Protection Act 2018 where relevant (Information Commissioner's Office, 2024). Converting digit groups to integers also removes leading zeros, so the output is numerically sorted but does not preserve the original textual formatting.

Environmental responsibility

Inefficient algorithms can consume unnecessary processing time and energy when applied to large data sets. Choosing an algorithm with better asymptotic efficiency can reduce computational work, although actual energy savings would need to be measured rather than assumed.

Professional reflection

This task reflects common legacy-code maintenance work: changes were made to data input and output while preserving the required C algorithm, and verification checks were

added to test the sorted result. Building checks into a program is more reliable than assuming that a successful run necessarily produced correct output.

9. Conclusion

Both tutor-provided programs were modified to read the CPUPI.txt data set, and the Python program was changed to use Timsort for the main sorting task. The report records 838,861 values and identifies sorted_data_full.txt as the output file; the file count and successful output should be checked against the actual submitted files.

The reported measurements show that Python's Timsort completed the full data set in 0.2394 seconds, compared with 711.551 seconds for C bubble sort. The results demonstrate why an O(n log n) algorithm can outperform an O(n^2) algorithm even when the latter is compiled. The estimated Python bubble-sort timings, single-run method and approximate 800,000-value Timsort result should remain clearly labelled.

The practical recommendations are to choose an algorithm suited to the data scale, use established library implementations where appropriate, and document test conditions while distinguishing measured results from estimates.

10. References

18. BCS, The Chartered Institute for IT (2022) *BCS Code of Conduct*. Available at: https://www.bcs.org/standards-and-professions/code-of-conduct/ (Accessed: 13 October 2026).
19. Bentley, J.L. and McIlroy, M.D. (1993) ‘Engineering a sort function’, *Software – Practice and Experience*, 23(11), pp. 1249–1265. doi: 10.1002/spe.4380231105.
20. Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to Algorithms*. 4th edn. Cambridge, MA: MIT Press.

21. Information Commissioner's Office (2024) *UK GDPR and data protection*. Available at: https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/ (Accessed: 13 October 2026).
22. Knuth, D.E. (1998) *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd edn. Reading, MA: Addison-Wesley.
23. Microsoft (2026) *Build C and C++ projects on the command line (MSVC)*. Available at: https://learn.microsoft.com/en-us/cpp/build/building-on-the-command-line (Accessed: 13 October 2026).
24. Python Software Foundation (2026a) *Sorting HOWTO*. Available at: https://docs.python.org/3/howto/sorting.html (Accessed: 13 October 2026).
25. Python Software Foundation (2026b) *Built-in Types – list.sort()*. Available at: https://docs.python.org/3/library/stdtypes.html#list.sort (Accessed: 13 October 2026).
26. Python Software Foundation (2026c) *time – Time access and conversions*. Available at: https://docs.python.org/3/library/time.html (Accessed: 13 October 2026).